

Team Details

Team Name: init to Win It

Team Leader: Jatin Rajani

Member 1: Kalpana Singh

Member 2: Kritika Pandey

Member 3: Anukriti Tiwari

Submitted to: Himanshu Sir

Title

**Machine Learning-Based Algorithm Selection for Irregular
Three-Dimensional Packing in Additive Manufacturing**

Index:

- Overview of the project
 - Purpose and libraries used
 - Detailed explanation of every class and function
 - Full explanation of all extracted features
 - Dataset structure, metadata, and workflow
-

3D Packing Algorithm Selection — Data Generation Pipeline Documentation

1. Overview

This pipeline generates a **synthetic dataset** for studying and predicting the best **3D packing algorithm** for different geometric scenarios in **Additive Manufacturing (AM)**.

Each “scenario” consists of multiple randomly generated 3D objects that must be packed efficiently into a fixed build volume.

For each scenario, several different packing algorithms are simulated, and the one performing best (based on a fairness-adjusted score) becomes the **label**.

The geometric and statistical properties of the objects and scenarios become the **features** (inputs) for a future machine learning model.

2. Tools and Libraries Used

Library	Purpose
<code>numpy</code>	Numerical computations, random number generation.
<code>pandas</code>	Data handling and tabular structure (features and labels).
<code>trimesh</code>	3D geometry creation and analysis (volume, surface, convex hull, etc.).
<code>random</code>	Randomized generation of dimensions, rotations, and algorithm performance.
<code>os, json, time, logging</code>	File management, metadata handling, and logging errors.
<code>tqdm</code>	Progress visualization for long-running loops.
<code>dataclasses</code>	Simplifies data structure definitions for storing algorithm results.

These are standard scientific computing and data-handling libraries — essential for simulation-heavy tasks.

3. Code Architecture

The system is modular, divided into six major components:

Component	Purpose
<code>PackingResult</code>	Structured output of algorithm performance.
<code>GeometryFactory</code>	Generates randomized 3D shapes and assemblies.
<code>PackerBase</code> + subclasses	Simulates performance of different packing algorithms.
<code>RichExtractor</code>	Extracts numerical features from objects and scenarios.
<code>make_scenario()</code>	Combines geometry, algorithm results, and feature extraction for one scenario.

Dataset builder functions

Generates, saves, and tracks dataset chunks and metadata.

4. Data Class — PackingResult

```
@dataclass
class PackingResult:
    efficiency: float
    compute_time: float
    stability_score: float
    success: bool
    algorithm_name: str
```

Purpose:

Stores the outcome of a packing algorithm run.

Field	Description
<code>efficiency</code>	Packing density (0–1). Higher means tighter packing.
<code>compute_time</code>	Time (seconds) algorithm took to complete.
<code>stability_score</code>	Measure of how stable the packed objects are (0–1).
<code>success</code>	Boolean indicating whether the algorithm succeeded.
<code>algorithm_name</code>	Name of the algorithm (e.g., “genetic”, “nfp_based”).

This structure provides a clean, consistent way to handle algorithm results.

5. Geometry Generation — GeometryFactory

Purpose:

Creates random 3D objects of different primitive types for each scenario.

```
PRIMITIVES = ["box", "cylinder", "sphere", "icosphere", "cone"]
```

Key methods:

`_rand_scale(minn, maxx)`

Returns a random float in range `[minn, maxx]` — used for random sizes.

`_rand_rot()`

Generates a random 3D rotation matrix:

- Picks a random axis in 3D space,
- Picks a random rotation angle,
- Builds a rotation matrix using `trimesh.transformations.rotation_matrix`.

Ensures every object is rotated differently — mimics realistic orientation variability.

`make_one(build_vol)`

Creates one random object:

- Randomly chooses a primitive type.
- Assigns random parameters (radius, height, extents, etc.).
- Builds the mesh using `trimesh.creation`.
- Applies a random rotation transform.
- Returns `(mesh, type)`.

`make_set(n_objs)`

Creates multiple objects for one scenario:

```
objs, kinds = factory.make_set(n_objs)
```

Returns:

- **objs**: list of trimesh objects,
 - **kinds**: list of corresponding types (e.g., ["cone", "box", "sphere"]).
-

6. Algorithm Simulation — **PackerBase** and Subclasses

Purpose:

Simulate multiple packing algorithms with randomized but realistic performance.

PackerBase

Defines a base class for all algorithms with:

```
def pack(self, objs, build_vol) -> PackingResult:  
    raise NotImplementedError
```

Each subclass must override **pack()**.

Algorithms Defined:

Algorithm	Description	Efficiency Range	Time Range (s)	Stability Range
BLFGravity	Bin Layer Filling with gravity awareness	0.66–0.89	0.4–0.8	0.80–0.96
FFD	First-Fit-Decreasing (fast greedy baseline)	0.61–0.83	0.2–0.5	0.71–0.89
SA	Simulated Annealing (metaheuristic optimizer)	0.79–0.94	1.2–2.5	0.76–0.93
NFP	No-Fit Polygon based, best for convex shapes	0.81–0.95	0.6–1.4	0.72–0.91
Genetic	Global optimization via evolution	0.85–0.97	3.5–6.0	0.73–0.92
LayerDecomp	Layer-based additive manufacturing method	0.77–0.92	1.8–3.5	0.82–0.97

All classes generate random values within realistic ranges and return a `PackingResult`.

The `PACKERS` registry holds all instances for easy iteration.

7. Feature Extraction — `RichExtractor`

Purpose:

Transforms raw 3D geometry into meaningful numeric features for ML models.

A. Object-Level Features

Extracted for each mesh in the scenario using `object_features(mesh)`.

Feature	Formula / Description
<code>volume</code>	Actual 3D volume of the object.
<code>surface_area</code>	Total surface area.
<code>bbox_volume</code>	Volume of the object's bounding box.
<code>packing_density</code>	Volume / bounding box volume — compactness ratio.
<code>sphericity</code>	$(36\pi \times \text{vol}^2) / \text{area}^3$ — closeness to spherical shape.
<code>convexity</code>	Volume / convex hull volume — 1 = perfectly convex.
<code>aspect_ratio</code>	Max axis / Min axis — how elongated shape is.
<code>elongation</code>	Max axis / Mean axis — shape proportionality.
<code>face_count</code>	Number of faces in the mesh.
<code>vertex_count</code>	Number of vertices.
<code>edge_density</code>	Edges / Vertices — mesh complexity.
<code>face_area_cv</code>	Coefficient of variation in face areas.

<code>genus</code>	Approx. number of holes (topological complexity).
<code>com_height</code>	Average height (center of mass along Z).
<code>overhang_pro xy</code>	Fraction of vertices near the bottom (overhang indicator).

B. Scenario-Level Features

Aggregated from all objects using `scenario_features()`.

Feature	Description
<code>n_objects</code>	Total number of objects in the scenario.
<code>type_divers ity</code>	Unique shape types ÷ total objects.
<code>total_volum e</code>	Sum of object volumes.

Then, for selected numerical properties (`volume`, `packing_density`, `sphericity`, etc.), computes:

- Mean (`_mean`),
- Standard deviation (`_std`),
- Coefficient of variation (`_cv`).

Example: `volume_mean`, `volume_std`, `volume_cv`, etc.

This captures both **average characteristics** and **variation** among parts.

8. Scenario Creation — `make_scenario(seed)`

Purpose:

Generates **one full training sample** — including 3D objects, algorithm results, feature extraction, and best algorithm label.

Steps:

1. Set random seed for reproducibility.
2. Create 3–10 random 3D objects.
3. Run all packing algorithms → collect results.
4. Compute **normalized score** for each algorithm:
Score = efficiency * stability / sq. root (time/expected time)
→ Balances efficiency, stability, and runtime.
5. Pick the algorithm with the highest score → **best_algorithm**.
6. Extract object + scenario features.

Return a single dictionary:

```
{  
  "n_objects": 5,  
  "type_diversity": 0.6,  
  "volume_mean": 120.5,  
  ...  
  "best_algorithm": "genetic",  
  "algorithm_results": {...},  
  "seed": 42  
}
```

This is the **atomic unit of your dataset**.

9. Dataset Generation and Storage

The final section automates generating many scenarios, saving them, and maintaining metadata.

Constants

Variable	Meaning
<code>INCREMENT_SIZE</code>	How many new scenarios per run (e.g., 1000).
<code>CHUNK_SIZE</code>	Size of each chunk file (keep same as increment).
<code>BASE</code>	Folder path where dataset is stored.

Functions

`get_next_seed_and_chunk()`

- Reads `/scenarios/` folder,
- Determines next chunk ID and seed offset,
- Enables incremental generation.

`build_chunk_from_seed(start_seed, size)`

- Runs `make_scenario()` for each seed in range,
- Builds a list of scenario dictionaries,
- Handles failures gracefully.

`append_scenarios()`

Main controller function that:

1. Finds next chunk index and seed.
2. Generates a new batch of scenarios.
3. Converts them into a `DataFrame`.
4. Splits into:

- `X_chunk`: numeric features (inputs)
 - `y_chunk`: best algorithm (labels)
5. Appends them to:
- `features.csv`
 - `labels.csv`
6. Saves backup JSON (`chunk_XXXX.json`).
7. Updates and saves `metadata.json` with:
- `total_scenarios`
 - `features` list
 - `algorithms`
 - `class_counts`
 - `class_balance`
 - `last_updated` timestamp

This allows continuous dataset growth with proper tracking and reproducibility.

10. Metadata and Dataset Files

File	Description
<code>features.csv</code>	All numeric features (one row per scenario).
<code>labels.csv</code>	The best-performing algorithm name (target label).
<code>/scenarios/chunk_XXXX.json</code>	Full chunk data with algorithm results and seed values.

`metadata.json`

Summary info — total count, feature names, algorithm list, and class distribution.

11. Workflow Summary

- ① GeometryFactory → creates random 3D parts
- ② PACKERS → simulates algorithm performance
- ③ RichExtractor → extracts geometry-based features
- ④ make_scenario() → combines results into one record
- ⑤ append_scenarios() → appends 1000 records per chunk
- ⑥ metadata.json → tracks dataset stats and balance

After several iterations, you have:

- Thousands of scenarios,
- Complete feature table for ML training,
- Label file with best algorithms,
- Metadata and backup files.

12. Key Takeaways

- This system **automates realistic synthetic data generation** for algorithm selection.
 - Every scenario is reproducible (via seed).
 - Each algorithm is **fairly compared** using a **time-normalized performance score**.
 - Rich geometric features ensure that the ML model can learn relationships between shape complexity and optimal algorithm choice.
 - The pipeline is **incremental**, scalable, and well-documented — ideal for experimental research.
-

14. Summary (for oral explanation)

“This pipeline automatically generates a dataset for selecting the best 3D packing algorithm.

It randomly creates 3D parts using `trimesh`, simulates several packing algorithms, extracts rich geometric and scenario features, scores each algorithm, labels the best one, and stores everything in structured CSV and JSON files.

It's reproducible, incremental, and designed for training ML models that can predict the optimal packing algorithm given any geometric scenario.”